Phase 1 Static FJSP Mathematical Model

FJSP-AGV-RL Research Platform

May 31, 2026

1 Scope

This document describes the Phase 1 static flexible job shop scheduling problem (FJSP) model implemented by the repository. Transportation time, AGV routing, dynamic job arrivals, reinforcement learning, and graph neural network components are outside the Phase 1 scope.

2 Sets and Indices

Symbol	Meaning
J	set of jobs, indexed by j
O_j	ordered set of operations for job j, indexed by o
M	set of machines, indexed by m
A_{jo}	eligible machines for operation (j, o)

Operation (j, o) must precede $(j, o + 1)$ for every job j and operation position o where $o + 1 \in O_j$.

3 Parameters

Symbol	Meaning
p_{jom}	processing time of operation (j, o) on machine m
H	conservative scheduling horizon

The implementation uses a conservative horizon

$$H = \sum_{j \in J} \sum_{o \in O_j} \max_{m \in A_{jo}} p_{jom}.$$

This upper bound is not necessarily tight, but it is safe for the small CP-SAT baseline model.

4 Decision Variables

Here a and b denote operation identifiers such as (j, o). The explicit y variables are useful for a classical mixed-integer programming statement. The current OR-Tools implementation uses optional interval variables and `NoOverlap` instead of manually creating all y variables.

Symbol	Meaning
$x_{jom} \in \{0,1\}$	1 if operation (j,o) is assigned to machine m
$S_{jo} \geq 0$	start time of operation (j,o)
$C_{jo} \geq 0$	completion time of operation (j,o)
$C_{\max} \geq 0$	makespan
$y_{abm} \in \{0,1\}$	order variable for two operations a, b on machine m

5 Objective

The objective is to minimize the makespan:

$$\min C_{\max}.$$

6 Constraints

6.1 Machine Assignment

Each operation must be assigned to exactly one eligible machine:

$$\sum_{m \in A_{jo}} x_{jom} = 1, \qquad \forall j \in J,\ o \in O_j.$$

Machines outside A_{jo} are not valid choices and are never created as candidate assignments in the implementation.

6.2 Processing Time Link

If operation (j,o) is assigned to machine m, then its completion time must match its start time plus the selected processing time:

$$C_{jo} \geq S_{jo} + p_{jom} - H(1 - x_{jom}), \qquad \forall j, o,\ m \in A_{jo}.$$

6.3 Job Precedence

Operations within the same job must follow the given route order:

$$C_{jo} \leq S_{j,o+1}, \qquad \forall j \in J,\ o, o+1 \in O_j.$$

6.4 Machine Capacity

A machine can process at most one operation at a time. For any two distinct operations $a = (j,o)$ and $b = (k,q)$ that are both eligible for machine m, the disjunctive ordering is:

$$S_a + p_{am} \leq S_b + H(1 - y_{abm}) + H(2 - x_{am} - x_{bm}),$$

$$S_b + p_{bm} \leq S_a + H y_{abm} + H(2 - x_{am} - x_{bm}).$$

These constraints are active only when both operations are assigned to machine m. In the CP-SAT implementation, this logic is represented by optional intervals and a per-machine `NoOverlap` constraint.

6.5 Makespan Definition

The makespan must be no smaller than every final operation completion time:

$$C_{\max} \geq C_{j,last(j)}, \qquad \forall j \in J.$$

7 Feasibility Checker Mapping

The repository's feasibility checker validates the following model conditions:

- every operation appears exactly once;
- all job and operation ids are inside the instance range;
- every selected machine is eligible for the operation;
- recorded processing time matches the instance data;
- job precedence constraints are satisfied;
- no two operations overlap on the same machine;
- stored makespan equals the maximum recorded completion time.

8 Dispatching Rules

The strict dispatching rules use a common candidate tuple

$$(j, o, m, r_{jo}, s_{jom}, f_{jom}, p_{jom}),$$

where r_{jo} is the job ready time, s_{jom} is the earliest feasible start time on machine m, and $f_{jom} = s_{jom} + p_{jom}$ is the tentative finish time.

Rule	Operation selection	Machine selection
FIFO	smallest (r_{jo}, j, o)	earliest finish
SPT	smallest p_{jom} pair	selected with pair
EFT	smallest f_{jom} pair	selected with pair
MWKR	largest remaining route work	earliest finish
MOPNR	largest remaining operation count	earliest finish
Random	random ready pair	selected with pair

For MWKR, remaining route work is estimated as

$$W_{jo} = \sum_{q=o}^{last(j)} \min_{m \in A_{jq}} p_{jqm}.$$

For MOPNR, the ready job priority is its number of remaining operations:

$$N_{jo} = last(j) - o + 1.$$

9 Experiment Result Schema

Phase 1 experiment scripts write normalized CSV files under `experiments/results/`. Each row has the following fields:

The CSV layer is intentionally simple. It records what happened in each experiment without changing solver behavior or feasibility definitions. All Phase 1 experiment scripts use shared runner helpers that perform timing, call the solver, run the independent feasibility checker, collect failures, and copy optional solver metadata into the CSV record.

Field	Meaning
`experiment_id`	stable experiment identifier
`instance_name`	benchmark or toy instance name
`algorithm`	solver or dispatching rule name
`makespan`	objective value, empty if no schedule was returned
`feasible`	`PASS` or `FAIL` after feasibility checking
`runtime_s`	measured wall-clock runtime in seconds
`note`	solver status, error, or validation note
`objective_bound`	CP-SAT best objective bound, when available
`gap`	relative gap $(C_{\max} - LB)/\max(1, C_{\max})$, when available
`solver_wall_time`	CP-SAT internal wall time, when available

9.1 Batch Benchmark Entry Point

The batch experiment recursively discovers `*.fjs` files in stable path order. Each loaded static FJSP instance is evaluated with the selected Phase 1 heuristics and, by default, the CP-SAT reference solver. A configurable per-instance time limit bounds CP-SAT work on larger collections. Loader failures are retained as `LOAD` rows with `FAIL` status so that a malformed file cannot disappear silently from an experiment report.

10 CP-SAT Diagnostics

For CP-SAT results, the implementation stores three optional diagnostics in the common `ScheduleResult` object:

$$LB = \text{BestObjectiveBound}, \qquad gap = \frac{C_{\max} - LB}{\max(1, C_{\max})},$$

and the solver-reported wall time. For an optimal solution on the toy instance, the bound equals the makespan and the reported gap is zero. Heuristic results leave these fields empty.